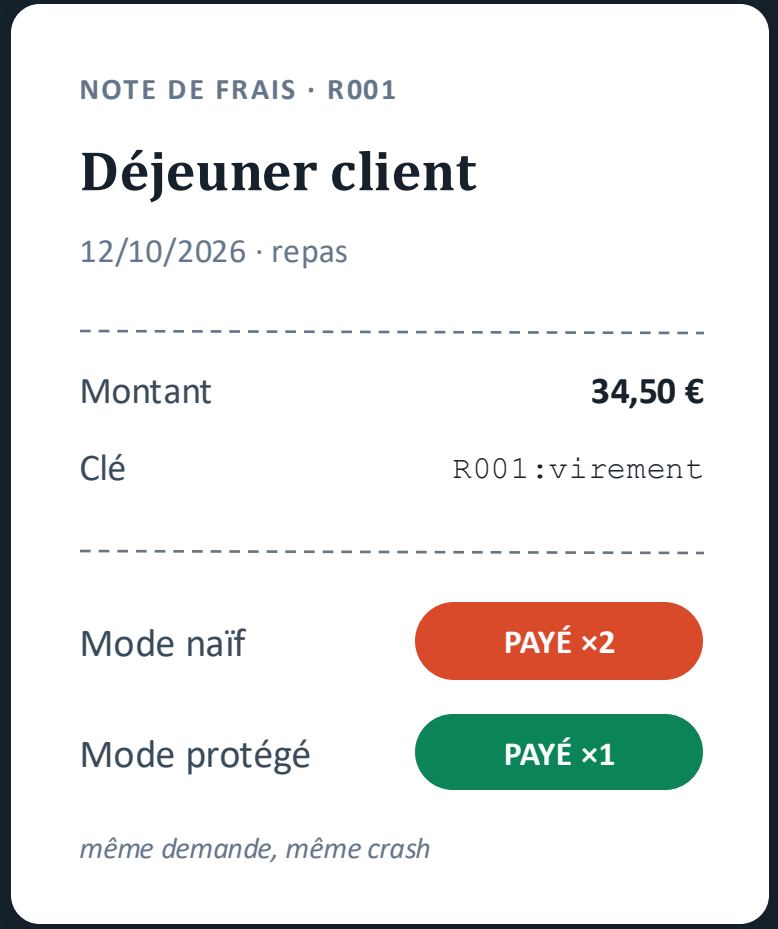
SOUTENANCE · IA GÉNÉRATIVE ET CHATBOTS

Notes de frais sous chaos

Un workflow agentique résilient et idempotent (sujet B5)

Groupe transformers · @nathan138 · @Marko075 · @neyhala
EPF · 28 octobre 2026



INTRODUCTION

Le problème en une phrase

Notre programme paie des employés. Que se passe-t-il s'il plante au mauvais moment ?

Objectif

Traiter chaque note, ne jamais payer deux fois, et le prouver par des mesures.



La même demande arrive deux fois

Une file de messages livre « au moins une fois ».



Le programme meurt en plein milieu

Crash, redémarrage, kill : à n'importe quel moment.



Un service répond en retard

Un timeout ne dit pas si le paiement a eu lieu.

Le scénario et les 5 étapes

Exemple de note *« Déjeuner client le 12/10/2026 : 34,50 € »*

1 — Réception
Validation et dédoublonnage par request_id

→

2 — Extraction
Le LLM transforme le texte en JSON

→

3 — Contrôle
Plafonds, dates, doublon métier

→

4 — Virement
Paiement simulé, clé d'idempotence

effet externe

→

5 — Notification
E-mail simulé, clé d'idempotence

effet externe

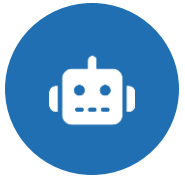
Les étapes 4 et 5 touchent le monde extérieur : aucun retour arrière en base ne peut les annuler.

Compensation : si la notification échoue, le virement est remboursé.

Le LLM : du texte libre au JSON validé

ENTRÉE · TEXTE LIBRE

« Déjeuner avec un client au restaurant le 12/10/2026 : 34,50 € TTC. »



Semantic Kernel → OpenRouter

Qwen3.8 Flash · température 0

SORTIE · JSON VALIDÉ

```
{
  "montant": 34.5,
  "devise": "EUR",
  "categorie": "repas",
  "date": "2026-10-12",
  "motif": "Déjeuner client"
}
```



Validé par un schéma

Devises et catégories autorisées, montant positif, date valide



Une tentative de réparation

Si le JSON est invalide ; sinon refus, sans paiement



Réponse mise en cache

Après un crash, l'appel n'est jamais refait

LLM réel et LLM simulé

	Vrai LLM	LLM simulé (mock)
Durée d'un appel	≈ 2,7 s	≈ 0,04 s
Coût d'un appel	≈ 0,0001 $	0
Réseau	nécessaire	aucun
Réponses	peuvent varier	toujours identiques
Pannes	subies	injectées à volonté

Même interface : le workflow appelle `llm.complete(texte)` et ne sait pas à qui il parle. Pour la démo et la campagne, le simulé suffit : on teste la fiabilité, pas le modèle.

PREUVE AVEC LE VRAI LLM

100 %

des champs corrects sur 10 notes réelles

11 / 150

appels réels consommés sur le budget

Crash forcé juste après l'appel : la réponse est relue dans le cache.

La fenêtre critique

FENÊTRE CRITIQUE



Virement envoyé

le worker appelle le service

Le service paie

paiement enregistré

Crash du worker

avant de noter « virement fait »

Relance

il a tout oublié

Il repaie

2 paiements pour 1 note

Même sans aucun crash

Le service paie, la réponse se perd (timeout) : un simple retry paie une deuxième fois.

Des retries sans idempotence créent des doublons.

Nos 3 mécanismes



Étapes sauvegardées

Chaque étape est enregistrée dans SQLite. Au redémarrage, on reprend à la première étape non terminée.

`table steps`



Clés d'idempotence

Chaque effet porte une clé stable. Le service la reconnaît et renvoie le reçu existant, comme Stripe.

`R001:virement`



Cache du LLM

Une extraction déjà faite est relue, jamais redemandée au modèle.

`llm_cache`

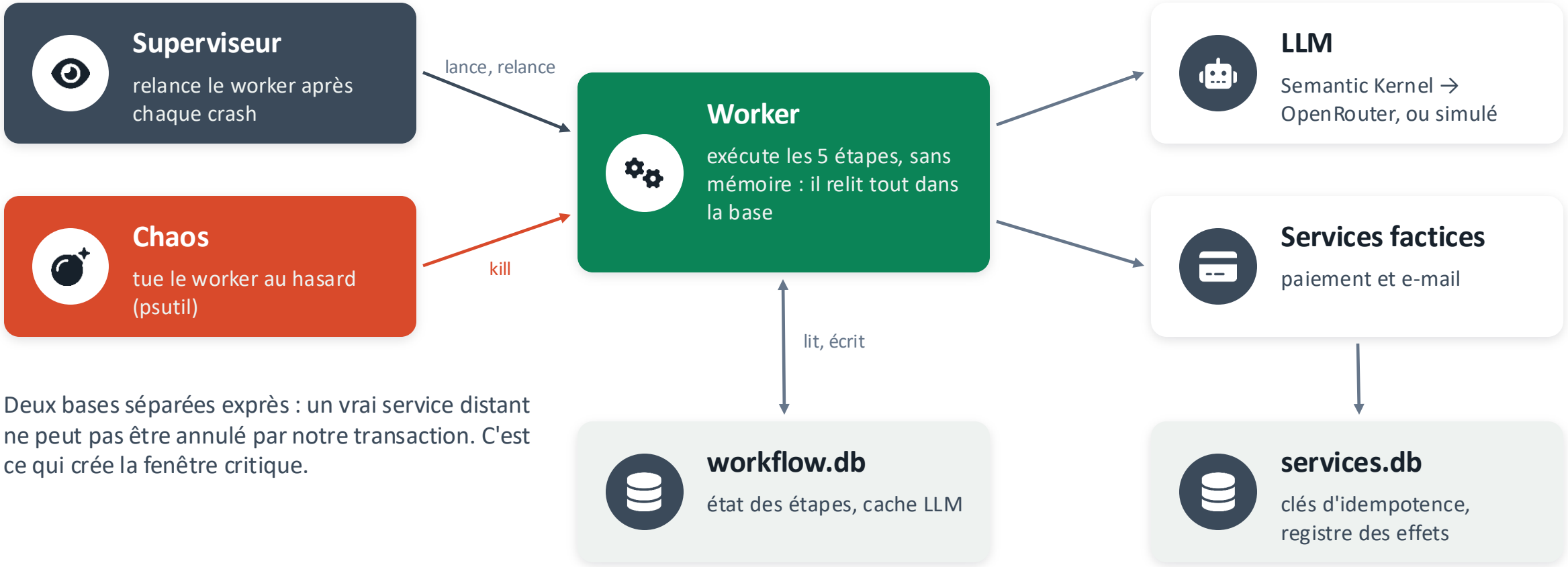


Compensation : remboursement si la notification échoue



Réconciliation : la clé dit si un paiement a eu lieu malgré l'échec

L'architecture



Deux bases séparées exprès : un vrai service distant ne peut pas être annulé par notre transaction. C'est ce qui crée la fenêtre critique.

Naïf contre protégé : la même demande sous crash

MODE NAÏF (BASELINE)

1. Crash pendant le virement
2. Relance : la demande repart de zéro
3. Le LLM est rappelé (coût en plus)
4. Nouveau virement, sans clé

Résultat : 2 paiements

MODE PROTÉGÉ

1. Crash pendant le virement
2. Relance : étapes 1 à 3 sautées
3. Virement refait avec la clé R001:virement
4. Le service répond « déjà payé »

Résultat : 1 paiement

Mêmes étapes, mêmes retries, mêmes pannes : seule la sémantique d'exécution change.

Le protocole de la campagne

40

notes synthétiques

28 normales, 9 refus attendus, 3 compensations

5 × 2

seeds × modes

naïf et protégé, même configuration

0,3–1,2 s

entre deux kills

le worker est tué au hasard

15 %

messages en double

la même demande livrée deux fois



Ce qu'on mesure

- Taux de complétion
- Effets en double, comptés dans le registre des services
- Demandes incohérentes : effets ≠ statut final
- Temps de reprise après crash (p50, p95)

Les résultats : 94 doublons contre 0

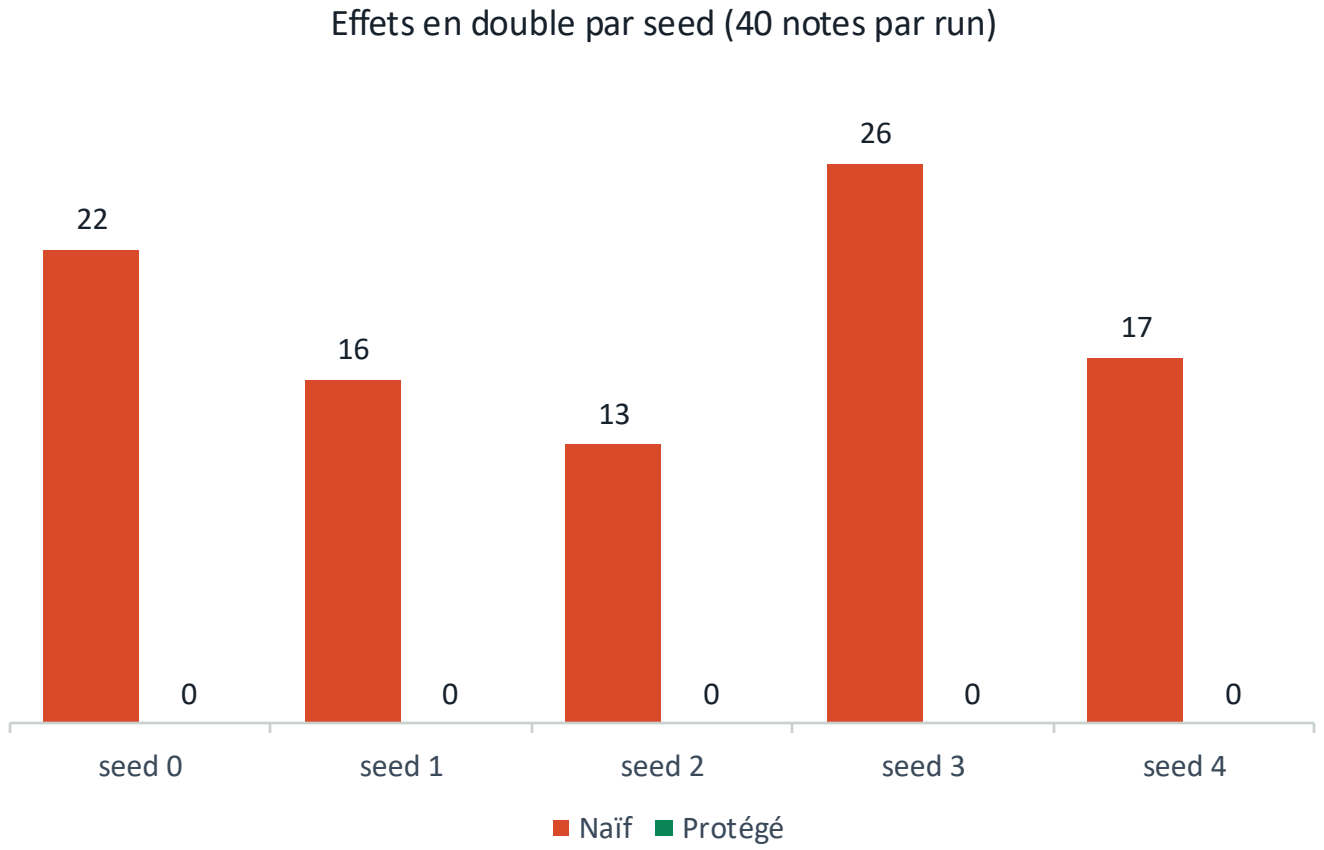
94

effets en double en mode naïf

0

effet en double en mode protégé

5 runs par mode	Naïf	Protégé
Crashs injectés	65	49
Demandes incohérentes	44 / 200	0 / 200
Étapes refaites	278	0
Reprise médiane	0,36 s	0,41 s



Complétion de 100 % dans les deux modes : le naïf « finit », mais dans un état faux.

Démo en direct

```
$ python -m demo
# naïf puis protégé, environ 25 s

$ python -m notesfrais dashboard
# tableau de bord dans le navigateur
```



Plan B : la vidéo de la démo enregistrée à l'avance, ou results/report.html, hors ligne.

CE QU'ON MONTRE

1. Les doublons du mode naïf en rouge, 0 pour le protégé
2. Une campagne lancée en mode protégé
3. Un clic sur « Tuer le worker maintenant »
4. Le worker redémarre et reprend la demande en cours
5. Le compteur « Effets en double » reste à 0

Les limites



Services simulés

En vrai, il faut un service qui accepte les clés d'idempotence (comme Stripe), sinon un registre côté client.



Un seul worker à la fois

Plusieurs workers en parallèle demanderaient un verrou par demande.



LLM mesuré sur 10 appels

Assez pour prouver le fonctionnement, pas pour juger finement la qualité d'extraction.



Reprise liée à la machine

Les 0,4 s mesurées sont surtout le démarrage d'un processus Python.

CONCLUSION

Ce qu'on retient

Réessayer ne suffit pas : il faut que refaire soit sans effet.

0

doublon sur 146 pannes injectées en mode protégé

PERSPECTIVES



Plusieurs workers en parallèle



De vrais services, type Stripe



De l'observabilité (OpenTelemetry)

CONCLUSION

Merci de votre attention

Vos questions ?

Code : dossier groupe-transformers
Démo : python -m demo

NOTE DE FRAIS · R001

Déjeuner client

12/10/2026 · repas

Montant	**34,50 €**
Clé	`R001:virement`
Mode naïf	**PAYÉ ×2**
Mode protégé	**PAYÉ ×1**

même demande, même crash